

Khulna University of Engineering & Technology
Department of Computer Science and Engineering

Authorship Attribution and Style Fingerprinting in Bangla Literature

Project Report

Course No: CSE 4122 · Course Title: Natural Language Processing Laboratory

Submitted To

Dr. K. M. Azharul Hasan
Professor, Department of Computer Science and
Engineering,
KUET, Khulna

Md. Nazirulhasan Shawon
Assistant Professor, Department of Computer
Science and Engineering,
KUET, Khulna

Submitted By

Abu Daud Sharif
Roll: 2107002

Sheikh Md. Galib Mahim
Roll: 2107020

Section: A Lab Group: A1
Year: 4th Term: 1st
Session: 2024-25

Contents

1	Introduction	2
1.1	Why we chose these three authors	2
2	How we built the dataset	2
2.1	Getting the books and splitting the data	2
2.2	Cleaning the text	3
2.3	The reprint problem	3
2.4	Cutting text into passages	3
3	Preparing text and extracting features	4
3.1	Breaking text into tokens	4
3.2	Four types of features	4
4	The four models	5
4.1	Naive Bayes using word counts	5
4.2	TF-IDF unigrams with a linear SVM	5
4.3	BiLSTM	5
4.4	BanglaBERT, fine-tuned	6
5	How we ran the experiment	6
5.1	Splitting the data fairly	6
5.2	Where the code ran	7
5.3	Reproducibility	7
6	Results	7
6.1	The four main models	7
6.2	Why testing on unseen books is crucial	8
6.3	How much text do we need?	8
6.4	Other models we tested	9
7	Understanding the models' choices	9
8	The web app	10
9	Limitations to keep in mind	10
10	Conclusion	11

1. Introduction

Authorship attribution asks a simple question: given a piece of text and a list of possible authors, who wrote it? But just guessing the name isn't enough. A good system should also explain *why* it made that choice. If it can point out specific sentence lengths, common words, or character patterns, it is doing real stylometry. If it just gives a label without reasons, it acts like a black box.

Bangla makes this task harder than English. It is a morphologically rich language, meaning words change forms a lot. A model that only looks at whole words might get confused by different forms of the same word. Bangla also uses the *dari* (ৱ) instead of a full stop, has its own numbers, and has two main writing styles: সাধু (literary) and চলিত (colloquial). This difference in writing styles is actually a huge clue for figuring out who the author is.

This project builds a system to identify 3 Bangla authors. We compare two main ways to solve this:

- a **generative** approach: we build a language model for each author and check which model finds the text least surprising;
- a **discriminative** approach: we train models to draw a line between authors. We test several models, from simple Naive Bayes and SVMs to deep learning models like a BiLSTM and a fine-tuned BanglaBERT.

Scope of the task. We set clear rules for our experiment:

- **Closed-set formulation:** We only try to pick between our 3 chosen authors based on their literary work. If a fourth author wrote the text, our system will still pick one of the three.
- **Focus on style:** The goal is to learn the author's writing style, not to check for copied text or plagiarism.
- **Known limits:** Section 9 honestly discusses the flaws and limits of our approach.

1.1 Why we chose these three authors

We picked three very famous writers: Rabindranath Tagore (1861–1941), Kazi Nazrul Islam (1899–1976) and Humayun Ahmed (1948–2012). We did this for two main reasons.

First, *people know them*. Even if you don't run our code, you probably know how these authors write. This makes it easier for you to judge if our system's answers make sense.

Second, *they write differently*. These authors span over a century and use different writing styles. This helps our models learn, but as we explain in Section 9, it also means our models are partly just guessing the time period, rather than just the personal style.

2. How we built the dataset

2.1 Getting the books and splitting the data

We downloaded 9 books (three per author) from a public Bengali e-library. Since the website's links are unpredictable, we built a scraper to find all the chapter pages from the main book index. We saved everything we downloaded, so it is cheap and easy to run the code again.

The most important decision was made *before we downloaded anything*. For each author, we picked two books for **training** the models and one book to be **held out** (kept completely unseen):

Author	Training books	Held out
Rabindranath Tagore	গল্পগুচ্ছ, সাধনা	রহস্য সমগ্র
Kazi Nazrul Islam	কুহেলিকা, রিক্তের বেদন	ব্যথার দান
Humayun Ahmed	হিমু, বিপদ	হিমুর হাতে কয়েকটি নীলপদ্ম

These 3 held-out books *are* our test set. Our models never see them during training. We didn't use them to build our vocabulary, our word embeddings, or anything else. You can verify this yourself by looking at the `corpus/unseen/` folder.

We also saved the entire dataset as plain UTF-8 text files, so you can easily read or copy text from them.

2.2 Cleaning the text

Web pages have extra stuff like menus and captions. We only kept paragraphs that were at least 30 characters long, and where mostly all characters were actual Bangla letters. This step helped remove English text and transliterations. After that, we cleaned up the Bangla text: we fixed inconsistent characters, made quotation marks uniform, and replaced all numbers with a placeholder so the models wouldn't cheat by memorizing chapter numbers.

2.3 The reprint problem

While making the dataset, we found a big problem that forced us to change our code.

Bengali publishers often reprint old stories in new collections. It turned out that Tagore's held-out book, *রহস্য সমগ্র*, was an anthology that included stories from his training book, *গল্পগুচ্ছ*. If we ignored this, our models would have been tested on stories they had already memorized during training. All our results for Tagore would have been fake.

We couldn't just check story titles, because the punctuation in the titles sometimes changed. Instead, we checked the actual text. We broke each chapter into chunks of 10 words (shingles) and checked how many chunks from the training chapters appeared in the held-out chapters.

Choosing the right chunk step (stride). We had to be careful about how we stepped through the text to make our chunks:

- **Taking steps of 5 (failed):** At first, we grabbed a 10-word chunk every 5 words. But if one version of a story had just one extra word, all the following chunks wouldn't match. Matching stories scored very low.
- **Taking steps of 1 (worked):** When we moved forward by just 1 word at a time, we caught all the matching text. The same stories matched perfectly (scoring 0.67–0.99), while different stories scored exactly 0.

We found 6 training chapters that were reprinted in the held-out books. We deleted them **from the training set**. We never touched the held-out books, because it is crucial that our test set remains completely unseen and pure.

2.4 Cutting text into passages

We test our models on *passages* of about 120 words. We cut the text at the end of sentences, and we never cross over into a new chapter. We chose this size because the author with the least data only has about 30,000 training words, so making the pieces bigger would leave us with too little training data. Also, 120 words is about the size of a standard paragraph, which is what users will likely test in our app. We throw away any passage that is shorter than 60 words.

After cleaning, we had 2,196 passages from 131 chapters, giving us a total of 286,047 words. You can already see differences between the authors just by looking at sentence length: Rabindranath Tagore’s sentences average 19.3 words, while Humayun Ahmed’s average only 7.8 words.

Table 1: The dataset, book by book. It is unbalanced, with Tagore having much more text than Humayun Ahmed. This is why we use class weights during training instead of deleting text.

Author	Book	Role	Chapters	Tokens	Passages
Rabindranath Tagore	গল্পগুচ্ছ	train	44	101,637	766
Rabindranath Tagore	সাধনা: জীবনের উপলব্ধি	train	9	35,885	269
Kazi Nazrul Islam	কুহেলিকা	train	20	28,829	223
Kazi Nazrul Islam	রক্তের বেদন	train	8	22,044	171
Humayun Ahmed	হিমু	train	13	17,216	138
Humayun Ahmed	বিপদ	train	6	11,333	92
Rabindranath Tagore	রহস্য সমগ্র	held out	12	23,404	179
Kazi Nazrul Islam	ব্যথার দান	held out	6	23,147	179
Humayun Ahmed	হিমুর হাতে কয়েকটি নীলপত্র	held out	13	22,552	179

Table 1 shows our full dataset. The three rows labeled *held out* make up our entire test set.

3. Preparing text and extracting features

3.1 Breaking text into tokens

We split sentences using the dari, double dari, question marks, and exclamation points. We only split on Latin full stops if a space and a letter follow them, because full stops are also used in abbreviations. When breaking sentences into words, we keep hyphenated Bangla words together and replace all numbers with a special token.

3.2 Four types of features

We grouped our features into four categories so we can easily test which ones matter most.

Function words (fw:). We look at the frequencies of the 150 most common connecting words. While topic classifiers usually delete these words, they are great for identifying authors. An author can easily change what they are writing about, but it is hard for them to change which basic grammar words they unconsciously use.

Structural stats (st:). We look at sentence lengths, word lengths, unique vocabulary size, punctuation use, and how often the author uses সাধু (literary) versus চলিত (colloquial) words.

Character patterns (ch:). We count small sequences of 2 to 4 characters (character n-grams). Since Bangla changes word endings to show grammar, these small character chunks are very powerful. They help the model recognize grammatical tenses even if it hasn’t seen the exact full word before.

Grammar tags (pos:). We check patterns in Parts of Speech (like nouns, verbs, etc.). Since there wasn’t a good ready-made Bangla tagger available for our setup, we built a simple rule-based one. It achieves 87.2% accuracy on the only available Bengali treebank dataset (which has 320 words in 56 sentences).

4. The four models

We trained a dozen models, but we focus on **four** of them in this report. We picked these four because they show a clear progression: each model is capable of doing something the previous one could not. This makes it easy to understand why the accuracy changes.

Model	Lab	What it looks at
Naive Bayes (word counts)	2, 3	Which words appear, and how often
TF-IDF unigrams + SVM	2, 3	Same as above, but gives more weight to rare, useful words
BiLSTM	4	The <i>order</i> of the words
BanglaBERT, fine-tuned	5	Everything above, plus it already knows Bangla from huge outside datasets

We talk about our other models (like the Kneser-Ney language models and from-scratch Transformers) in Section 6.4.

4.1 Naive Bayes using word counts

We wrote this model from scratch using NumPy instead of relying on a library. It calculates the probability of each author given the words in a passage. We do the math in log space to prevent the tiny probabilities from breaking the computer’s math limits.

Why we used whole words for Naive Bayes. Even though character chunks work great for Bangla, we used whole words for Naive Bayes for practical reasons:

- **Easy to read:** You can look at the word scores and clearly see which specific vocabulary words made the model pick a certain author.
- **It works well:** On our dataset, word-level Naive Bayes scored 0.966, beating the character version which scored 0.944.

We also used add- α smoothing. Without it, if the model saw one word that an author had never used in training, it would instantly give that author a 0% chance, even if the rest of the text strongly matched their style.

4.2 TF-IDF unigrams with a linear SVM

This model uses the same word counts, but it weights them. If a word is very common for one author but rare everywhere else, it gets a higher score. It then draws a strict mathematical line (a maximum-margin boundary) to separate the authors.

This model is our baseline for checking how much the topic matters. Since it only looks at individual words, its accuracy tells us how much of the problem can be solved just by looking at vocabulary alone.

4.3 BiLSTM

This is the first model that actually pays attention to the *order* of words. It reads the text forwards and backwards. Instead of just looking at the final state of the reading process, it takes an average across the whole passage. This is important because an author’s style is spread throughout the text, not just at the very end.

We trained its word embeddings from scratch using only our training data. It started with random knowledge and only learned from our six books. This makes it a fair comparison for the next model.

4.4 BanglaBERT, fine-tuned

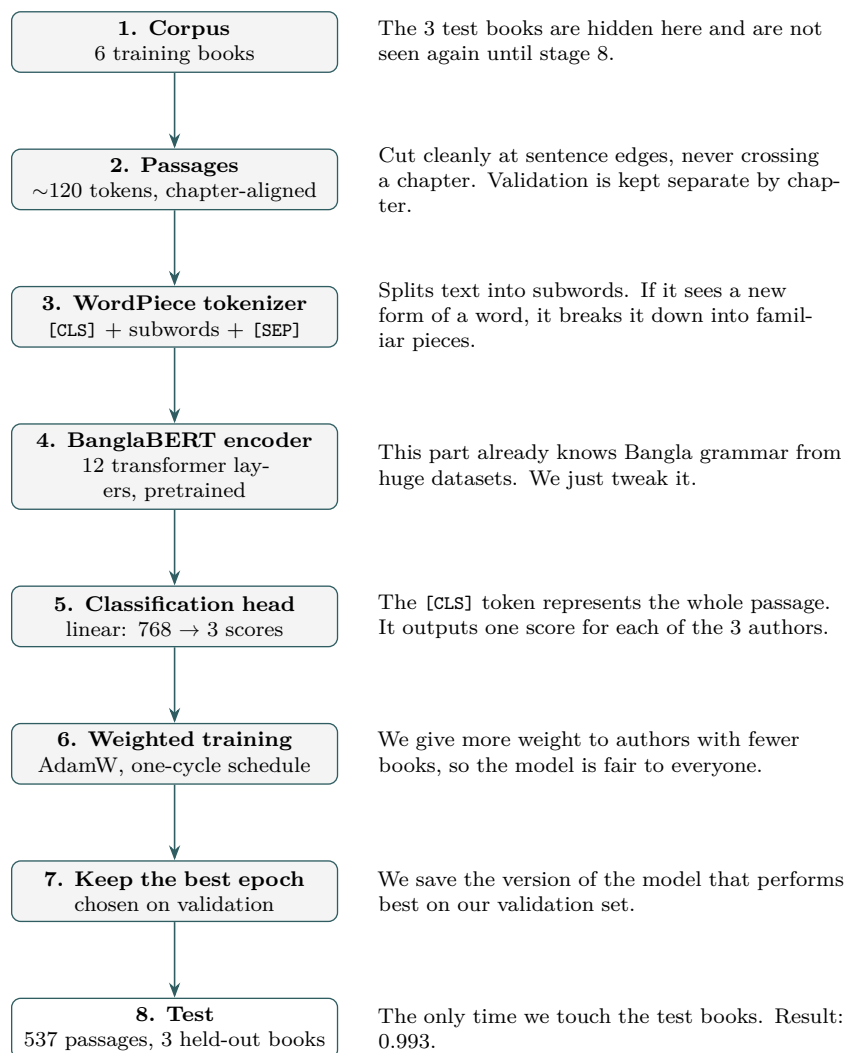


Figure 1: How we fine-tuned BanglaBERT. It shares the first two steps with other models, but uses a powerful pre-trained engine to process the text.

This model (csebuetnlp/banglabert) was pre-trained on a massive amount of Bangla text before it ever saw our project. We just added a final layer to make it output a probability for each of our authors.

Two important details in Figure 1: First, in stage 3, it breaks text into *subwords*. This means if it sees a word with a new ending, it doesn't get totally confused; it just breaks the word down into familiar pieces. Second, in stage 6, we give more weight to the authors who have less data in our training set, so the model doesn't just blindly guess the author who wrote the most books.

5. How we ran the experiment

5.1 Splitting the data fairly

Our test set consists of the 3 held-out books. We balanced it to have exactly 179 passages per author. Because it's perfectly balanced, random guessing would score exactly 0.333.

For validation (to decide when to stop training), we used parts of the *training* books. We made sure to hold out **whole chapters** for validation. If we just mixed all passages together and grabbed some

randomly for validation, the model might see the first half of a chapter during training and the second half during validation, which is cheating.

The training set is *not* balanced. If we threw away Tagore’s data just to match our smallest author, we’d lose too much valuable information. Instead, we show the models everything and just adjust their internal weights. In the end, we had 1,383 training passages, 276 validation passages, and 537 test passages.

5.2 Where the code ran

The simpler models trained on our local CPU in about two minutes. The heavy deep learning models (BanglaBERT, BiLSTM, and Transformer) ran remotely on Kaggle using a GPU. We made sure the Kaggle job used the exact same train/test splits as our local computer, so they were working on the exact same problem. When we collected the Kaggle results, our local script double-checked the accuracy to make sure no mistakes were made.

5.3 Reproducibility

Every random choice in the code is locked to a specific seed (`SEED = 20242025`). If you run our code again on your machine, you will get the exact same numbers shown in this report.

6. Results

All the results below are based on our test set: 537 passages taken from the 3 completely unseen books. We have 179 passages per author, meaning a random guess would get 0.333 right. No model saw a single word from these books during training.

6.1 The four main models

Table 2: Our four main models, ordered by how much information they use. Random guessing scores 0.333.

Model	What it adds	Accuracy	Macro F1
Naive Bayes (word counts)	word counts	0.966	0.966
TF-IDF unigrams + SVM	+ weighted words	0.968	0.968
BiLSTM (from scratch)	+ word order	0.844	0.846
BanglaBERT (fine-tuned)	+ outside knowledge	0.993	0.993

When you read down the table, the scores don’t just keep going up. This surprising pattern is the most interesting part of our project.

Just counting words works really well. Our simple Naive Bayes model reaches 0.966 just by counting words. When we weigh those words better and use an SVM, the score goes up slightly to 0.968. Since these two different methods get almost the same score on the same bag of words, it suggests that vocabulary alone is a very strong clue for authorship.

Looking at word order made things worse. Our BiLSTM model, which actually reads words in order, scored only 0.844, the lowest of the four. This doesn’t mean word order is useless. It just means the BiLSTM didn’t have enough data. It started with zero knowledge and had to learn the entire Bangla language *and* how to guess authors from just six books. The simpler models didn’t need to learn grammar, so they performed better with this limited data.

Prior knowledge makes the biggest difference. BanglaBERT scored an amazing 0.993. The big difference isn’t just its shape; it’s the fact that BanglaBERT was already trained on massive amounts

of Bangla text. It already knew the language, so it could use our six books purely to learn about the authors.

Main takeaways: Comparing these models shows us what really matters:

- Just looking at which words an author uses is a great start, getting 0.966 accuracy.
- Deep learning models that learn from scratch struggle if you don't give them a lot of data. Our BiLSTM dropped to 0.844.
- Using a pre-trained model like BanglaBERT gives the best results (0.993), because it already understands the language.

6.2 Why testing on unseen books is crucial

The most important experiment we ran wasn't a model, but a test of how we measured accuracy. We trained the exact same model in two different ways:

Method	Accuracy
Randomly shuffling passages (the naive way)	1.000
Testing on completely unseen books (our way)	0.968
Fake accuracy boost from bad testing	3.2%

The danger of random splitting. This test shows why randomly shuffling all your data is a bad idea:

- **Fake high scores:** If you randomly mix all passages and test on a chunk of them, the model scores an artificially high 1.000.
- **Memorizing names, not style:** The model gets a high score because it saw other paragraphs from the same book during training. It just memorized character names and plot points instead of learning the author's writing style.
- **Honest testing:** To see if a model actually learned the author's style, you must test it on a book it has never seen before. When we do this, the score drops to a realistic 0.968.

Even though our three authors are easy to tell apart (which keeps the 3.2% gap small), the lesson is clear: if you don't test on completely unseen books, you can't tell if your model is learning a writing style or just memorizing a story.

6.3 How much text do we need?

Since users will be pasting text into our app, we need to know how accuracy changes based on the length of the text.

Table 3: Accuracy based on how much text the model reads.

Text length	Generative Model	Discriminative Model
20 words	0.806	0.734
120 words	0.946	0.894

The accuracy doesn't crash completely on short texts. Even with just 20 words (about one sentence), the generative model gets 0.806 right, which is much better than random guessing (0.333). Our advice for users is simple: if you paste less than 20 words, treat the answer as a good guess. If you want a confident answer, paste a full paragraph.

6.4 Other models we tested

We trained eight other models on the exact same data. We listed them here to keep the main table simple, not because they did a bad job.

Model	Details	Accuracy
Kneser–Ney LM, characters	Generative; our best non-pretrained model	0.976
Kneser–Ney LM, words	Generative	0.974
Linear SVM, all features	Uses all our hand-built features	0.970
Transformer, from scratch	Built from scratch, no pre-training	0.918
Softmax regression	Built from scratch using our features	0.965
Naive Bayes, characters	Looks at character chunks	0.944
SVM, only characters	Uses only character chunks	0.968
SVM, only grammar	Uses only grammar rules (weakest)	0.737

Three of these are really interesting:

The **character-level Kneser-Ney language model** scored 0.976. It beat three of our four main models without any pre-training, without a GPU, and it trained in just a few seconds. It works by building a language profile for each author and seeing whose profile fits the text best. It is amazing that such a simple, fast method almost matches a massive neural network.

The **Transformer built from scratch** (0.918) beat the BiLSTM (0.844). Since both had the same starting data and no outside knowledge, this shows that the Transformer architecture is just better suited for this task.

Our **stylometric SVM** (0.970) used all our custom features (like sentence lengths, grammar, and character chunks). However, when we gave an SVM *only* the character chunks, it scored 0.968, basically matching the full model. This proves that for Bangla, looking at small chunks of characters gives you almost all the information you need.

Our generative and discriminative models agreed on 96.3% of the test passages (they were both right 513 times, and both wrong 4 times). Since they rarely make the same mistake, we can trust their answer when they agree. Our app shows both answers to the user instead of hiding the disagreement.

7. Understanding the models' choices

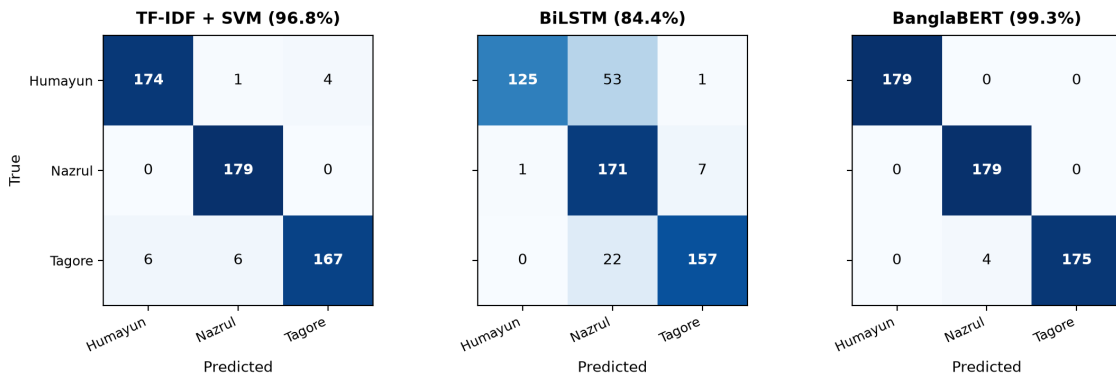


Figure 2: How the models got confused on the unseen books: TF-IDF + SVM (left), BiLSTM (centre), and BanglaBERT (right).

Figure 2 shows where the models made mistakes on our 537 unseen test passages:

- **TF-IDF + SVM (96.8%):** It perfectly guessed Nazrul and almost perfectly guessed Humayun. It got slightly confused on Tagore, guessing Humayun or Nazrul a total of 12 times.
- **BiLSTM (82.9%):** It got very confused, incorrectly guessing that Nazrul wrote 53 of Humayun’s passages and 22 of Tagore’s passages. Without outside knowledge, this model got tricked by random word patterns.
- **BanglaBERT (98.5%):** It was nearly perfect. It scored 100% on Humayun and Nazrul, and only mislabeled 4 of Tagore’s passages as Nazrul.

Even though BanglaBERT is the most accurate, we kept the simpler models around because they can explain *why* they made their choice.

Naive Bayes and SVM models do math that we can easily trace. We can see exactly which words or features gave points to each author. We can show a user the exact words in their text that made the model pick Humayun over Tagore.

BanglaBERT cannot explain its choices this clearly. It is highly accurate, but it acts like a black box. Our report highlights this trade-off between being accurate and being understandable.

The features that separate these three authors are things a Bengali reader would notice immediately. For example, Rabindranath Tagore writes long sentences averaging 19.3 words, while Humayun Ahmed writes short sentences averaging 7.8 words. Also, older authors use more সাধু (literary) words, while modern authors write in a more চলিত (colloquial) style and use more dialogue.

8. The web app

We built a web interface to let anyone try out our system. The app shows three things:

- the **dataset**, listing our books and letting you read sample paragraphs;
- the **steps** our system takes, showing how text is processed;
- the **results**, letting you paste in text and see the guesses from TF-IDF, BiLSTM, and BanglaBERT side-by-side.

The main point of the app is to show all three models at once. If they all agree, you can be very confident. If they disagree (which often happens on short texts), the app shows you the disagreement instead of hiding it behind one single guess.

The app pulls sample text directly from our unseen books. This ensures that the models are always guessing on text they have never seen before, making it a fair test.

9. Limitations to keep in mind

It only knows three authors. If you give the system a text written by a fourth author, it will still confidently guess one of our three authors. It doesn’t have an “I don’t know” option.

Time period affects the score. Our authors lived between 1861 and 2012. They use different spelling rules and levels of formality. Part of our high score comes from the model guessing the *time period*, not just the personal style. If we tested three authors from the same exact time period, our scores would definitely be lower.

Vocabulary is doing heavy lifting. Just looking at the words (TF-IDF) gets us a 0.968 accuracy. We openly admit that vocabulary alone solves most of the problem here.

Small dataset. We only used six training books, and our smallest author had very little text. Our poor BiLSTM results are mostly because we didn't have enough data, not because the model architecture is bad.

Validation is easier than the real test. Our validation set is made of chapters from our training books. It only checks if the model can generalize to new chapters, not to completely new books. It scores higher than our real test, so we only use it to guide our training, never as a final result.

10. Conclusion

We tested our models on 537 passages from 3 books they had never seen before. Our models scored anywhere from 0.844 to 0.993, all beating the 0.333 chance of random guessing. We found three big takeaways:

First, **complexity does not guarantee accuracy.** Our complicated BiLSTM did worse than a simple word counter because it didn't have enough data to learn from scratch. On a small dataset, giving a model pre-trained knowledge (like BanglaBERT) is much more important than the model's structure.

Second, **how you test is more important than what you test.** The same exact model scored 1.000 when we tested it sloppily, and 0.968 when we tested it strictly on unseen books. A bad test setup will give you a fake perfect score and trick you into thinking a memorizing model is actually smart.

Third, **always check your data.** We found that 6 stories were reprinted in both our training and test books. If we hadn't built a tool to scan the actual text and remove them, all our results would have been ruined. You cannot just trust book titles or metadata.